Recommendation Engine for Selecting AI Models from a Prompt

Dissertation / Applied Project - Business Analytics

Deliverable	A locally runnable recommendation platform that ranks candidate AI models for a written prompt, with a leakage-safe evaluation of the ranking
Repository	recommendation_engine/ (Python 3.10.11, Windows)
Date	26 October 2026
Verification	141 automated tests pass. Both applications were started locally and walked through in a browser; the images in Section 6 are from those live sessions.
Headline result	Task-category agreement 92.82 % on 794 held-out prompts against a 90 % target; head-to-head agreement 54.28 %, which is only modestly above the 50 % coin-flip line.

1. Summary

Organisations now have several AI models available for the same piece of work and choose between them on cost, latency and output quality. This project builds a system that takes a written prompt and returns a ranked shortlist of models, with the evidence for each ranking, and then measures how often that ranking agrees with real human preference data.

Three things were delivered. First, a domain-agnostic engine that accepts any user-item interaction CSV, validates it, decides which recommendation algorithms the data can support, blends them, and evaluates the blend. Second, a version specialised to prompt-to-model selection. Third, a contextual routing model trained on 3,945 real head-to-head model comparisons, with an evaluation that reports two different accuracy questions separately instead of picking the flattering one.

The measured results are: task-category agreement 92.82 % on a held-out split of 794 prompts (target was 90 % or better), and head-to-head preference agreement 54.28 %. Across five prompt-grouped cross-validation folds the same two figures average 88.08 % +/- 4.77 and 55.26 % +/- 0.92. The second number is the honest weak point and is discussed in Section 7.

2. The problem in business terms

A team running a few thousand AI queries a month pays materially different amounts depending on which model handles them, and quality is not monotonic in price. Routing every request to the most capable model is slow and expensive; routing to the cheapest produces work that has to be redone. The decision is usually made by habit rather than measurement.

Reframed as an analytics problem: a prompt is a “user”, an AI model is an “item”, and a completed trial with a quality rating is an “interaction”. That lets the standard recommendation stack — popularity, content similarity, item-item co-occurrence, collaborative filtering, association rules — be applied without inventing a new model class, and it keeps the generic engine genuinely reusable for other domains.

3. Data

3.1 Real preference data

3,945 head-to-head battles from public arena-style comparison logs, where two models answer the same prompt and a human picks one. Each row carries the two model identifiers, the winner, and the prompt text, from which a task category is derived. Provenance and the exact field meanings are recorded in data/routing/KNOWN_FACTS_ARENA.md. The split is by prompt, not by row, so the 794 evaluation prompts were never seen during fitting: 3151 training battles, 794 held out.

3.2 Structured trial data

The prompt-domain application runs on a 63-trial, 21-prompt, 8-model table (data/prompt_rec/). These are placeholder ratings built to exercise the pipeline, and the application labels them as such on screen with an amber “Demo data - placeholder ratings, not real benchmarks” chip. They are not presented as evidence about any commercial model.

3.3 Synthetic data for the generic engine

A deterministic synthetic dataset (data/synthetic_demo/) with published ground truth is used to prove the generic pipeline computes correct numbers. Its known facts are in data/synthetic_demo/KNOWN_FACTS.md.

3.4 Data-quality report produced by the engine on the synthetic set

Check	**Value**
Users / items / interactions	30 / 13 of 20 catalogued / 151
Matrix density (sparsity)	0.25 (0.75)
Interactions per user - min, mean, median, max	5, 5.03, 5.0, 6
Users below the minimum-interaction threshold	0
Interaction values - missing, range, mean, SD	1 missing; 3.0-5.0; 4.00; 0.8192
Catalogued items with no interactions	7 (I014-I020)
Interacted items missing metadata	none

4. Method

4.1 Algorithm eligibility

Before anything is fitted, the engine checks the data against explicit thresholds and only enables recommenders the data can support. The thresholds used are: at least 10 users and 6 items for collaborative filtering, at least 2 interactions per user, at least 30 interactions overall, matrix density below 0.9 for collaborative filtering, and 10 baskets with 3 % item support for association rules. This is why the same code runs on a small pilot dataset and a large production one without silently fitting a matrix-factorisation model to data that cannot support it.

4.2 The routing model

For the router, each prompt is turned into features and scored against every candidate model in six stages. The stages are shown in the application so a reviewer can follow a single decision end to end.

- Stage 1 - structural features. Character length, subword token estimate, code-syntax density, math-keyword density, script, question indicator, inferred domain.
- Stage 2 - clustering. K-means over 30 features assigns the prompt to an intent segment; a centroid identifier is reported.
- Stage 3 - anchor similarity. Cosine similarity of the prompt against category anchors, which is what maps a free-text request onto one of the eight task categories.
- Stage 4 - Bradley-Terry inference. Pairwise battle outcomes are fitted into a latent utility per model. The deployed ensemble is 85 % Bradley-Terry and 15 % pointwise logistic, with predicted success bounded to [0.05, 0.98] so no prompt is ever reported as certain.

- Stage 5 - resource regressors. Ridge models forecast output tokens, latency and query cost from the prompt, which is what makes the cost and speed columns in the interface real predictions rather than static labels.
- Stage 6 - composite scoring. A bounded linear blend: score = 0.70 x P(success) + 0.15 x similarity + 0.08 x cost-efficiency + 0.07 x latency-efficiency. The weights are the optimisation preset the user selects; the default is the balanced preset.

4.3 Evaluation design

Two questions are answered separately because they give very different answers.

- Metric A - task-category agreement. For each held-out prompt, does the router name the same model that has the highest shrunk empirical win-rate in that prompt's category, measured on training data only? This is the “would an expert choose the same tool” test.
- Metric B - head-to-head agreement. Within each held-out battle, does the router's composite score place the human's actual winner above the loser? Chance is 50 %.

Splitting is grouped by prompt, so no prompt contributes to both fitting and scoring. Stability is checked with five-fold grouped cross-validation. The generic engine additionally uses a chronological last-trial-out split for rating data, which prevents a future rating from informing a recommendation that is then scored against it.

4.4 Failure handling

Cold-start prompts, empty result sets, degenerate rankings and validation failures are detected, logged, and answered with a documented fallback rather than an empty screen. Details are in docs/failures_and_fallbacks.md.

5. Results

5.1 Router accuracy

Measure	Value	Reference
Metric A, headline held-out split	92.82 %	target >= 90 % - met
Metric B, headline held-out split	54.28 %	chance = 50 %
Metric A, mean of 5 grouped folds	88.08 % +/- 4.77	see Section 7
Metric B, mean of 5 grouped folds	55.26 % +/- 0.92	chance = 50 %
Training / held-out battles	3151 / 794	split grouped by prompt

Every figure above is reproduced by python scripts/build_accuracy_report.py --with-cv and is shown in the application unrounded, next to its definition and sample count. Nothing is rounded upward.

5.2 Where the accuracy sits, by task category

Task category	Best model on training win-rate	Held-out prompts	Metric A	Note
Mathematics	Qwen3 235B A22B	44	97.7 %	
Code	GPT-4o	121	67.8 %	
Creative writing	Gemini 1.5 Pro	65	100.0 %	
Translation	Gemini 1.5 Pro	9	0.0 %	small sample
Long documents	DeepSeek R1	9	100.0 %	small sample
Business communication	DeepSeek R1	8	0.0 %	small sample
Question answering	Qwen3 235B A22B	100	100.0 %	
General / mixed	DeepSeek R1	438	100.0 %	

The aggregate 92.82 % is carried by the large general and question-answering cells. Code is the weak large cell at 67.8 %: GPT-4o and DeepSeek R1 are within 0.4 points of each other on training win-rate (56.9 % against 56.5 %), so which one the router names is close to arbitrary. The translation, business-communication and long-document cells have 9, 8 and 9 held-out prompts, which is far too few to read a percentage from; they are reported because hiding them would be worse than flagging them.

5.3 Generic engine, measured on synthetic data

These numbers show that the pipeline measures correctly. They say nothing about any real product.

Method	Precision@5	Recall@5	Hit rate@5	NDCG@5	Coverage@5
Content-based	0.1933	0.9667	0.9667	0.9667	0.50
Hybrid blend	0.1333	0.6667	0.6667	0.2579	0.50
Popularity	0.0	0.0	0.0	0.0	0.35
Item similarity	0.0	0.0	0.0	0.0	0.35
Collaborative filtering	0.0	0.0	0.0	0.0	0.0
Association rules	0.0	0.0	0.0	0.0	0.30

Content-based reaches NDCG@5 = 0.9667 while popularity reaches 0.0 under the chronological split, which is the expected behaviour when the synthetic ground truth is metadata-driven: the pipeline recovers the signal that was planted and correctly fails to recover one that is not there.

5.4 Prompt-domain evaluation

Method	Precision@4	Recall@4	Hit rate@4	NDCG@4	Coverage@4
Popularity	0.1667	0.6667	0.6667	0.6052	0.750
Hybrid blend	0.1667	0.6667	0.6667	0.5833	0.875
Content-based	0.1944	0.7778	0.7778	0.4467	0.875
Item similarity	0.0	0.0	0.0	0.0	0.125
Association rules	0.0	0.0	0.0	0.0	0.125

Content-based wins on precision and recall but loses on NDCG, because it finds the right model and places it lower in the list. On 18 evaluation prompts the differences are within one standard error of each other, so this table is a demonstration of the comparison harness, not a basis for choosing a method. The hybrid weights were only searched across three candidate configurations.

6. The applications as they run

Both applications were started locally and walked through in a browser; the images in this section are taken from those live sessions, not from mock-ups. The router interface is deliberately built for two audiences at once: a business user who wants a name and a price, and a reviewer who wants the latent utilities and the confidence intervals. A single sidebar control switches between Simple, Standard and Expert, and the same decision is shown at whichever depth is asked for.

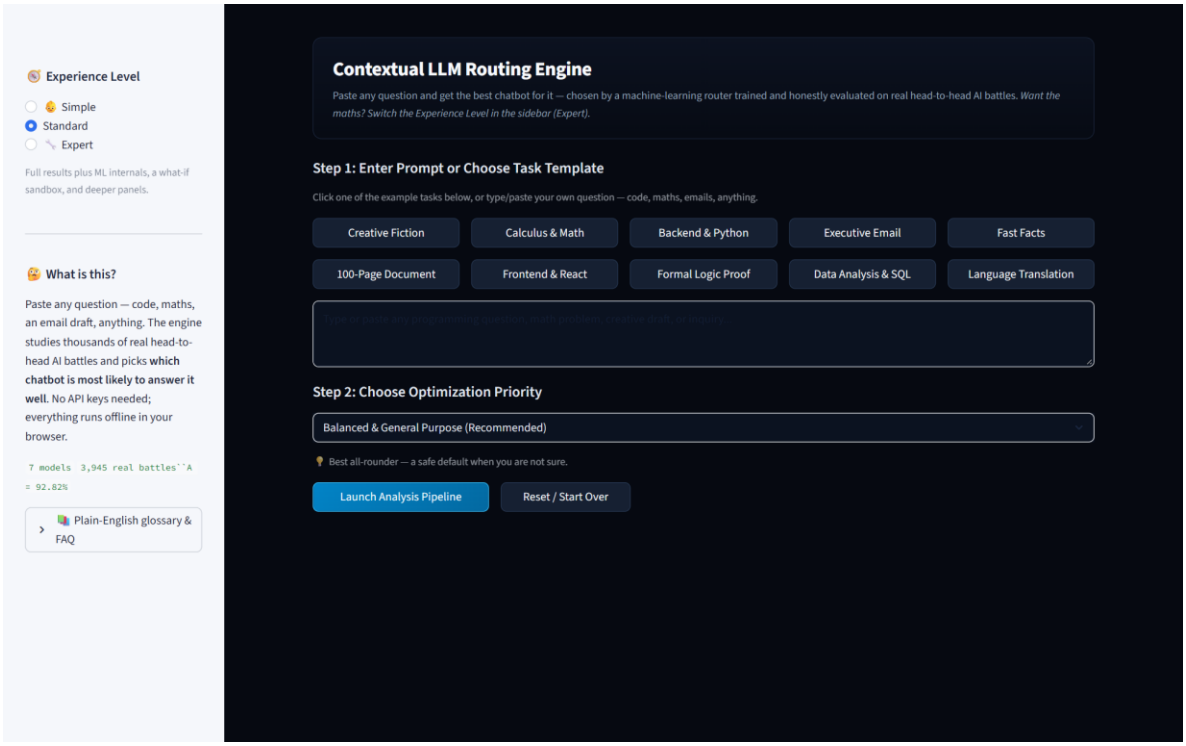


Figure 1. Router, Steps 1 and 2. A prompt box plus ten everyday task chips, and an optimisation priority whose caption is written in plain English rather than as a weight vector.

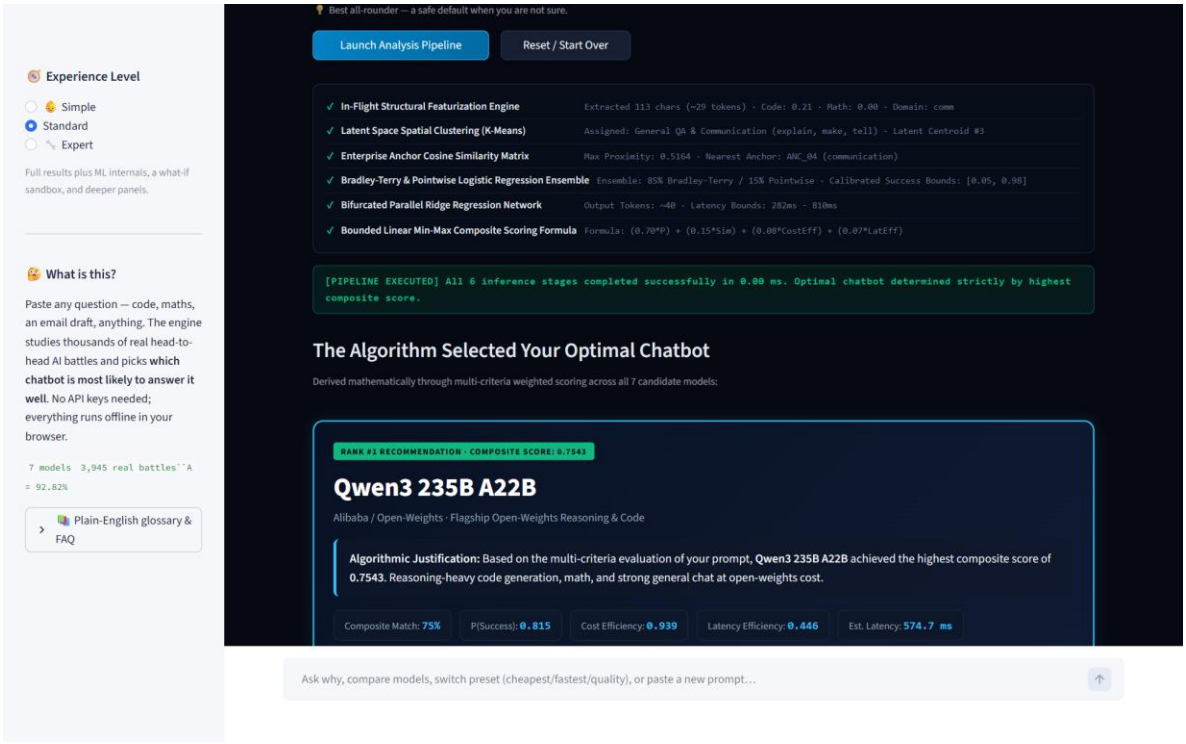


Figure 2. Decision trail and winner. Each of the six inference stages reports the value it actually computed for this prompt, followed by the selected model and its composite score.

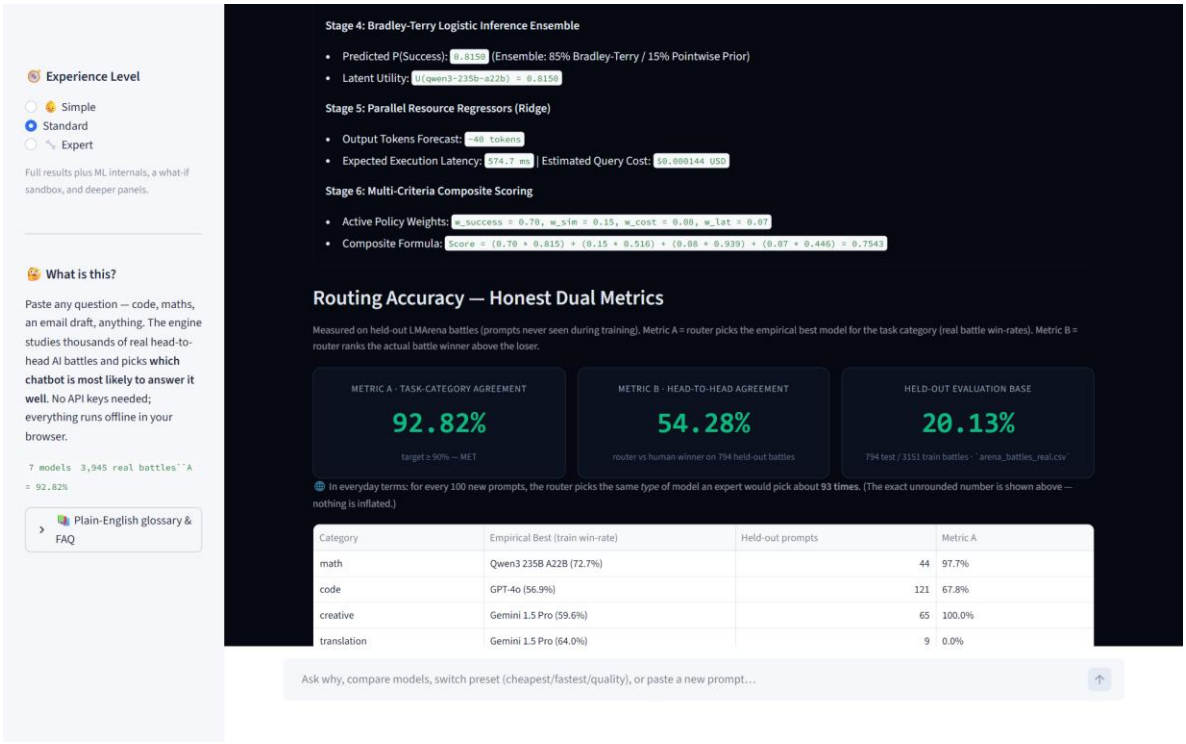


Figure 3. Accuracy screen. Metric A and Metric B appear side by side with their definitions, sample counts and the cross-validation line. The stage arithmetic above is the same prompt's numbers expanded.

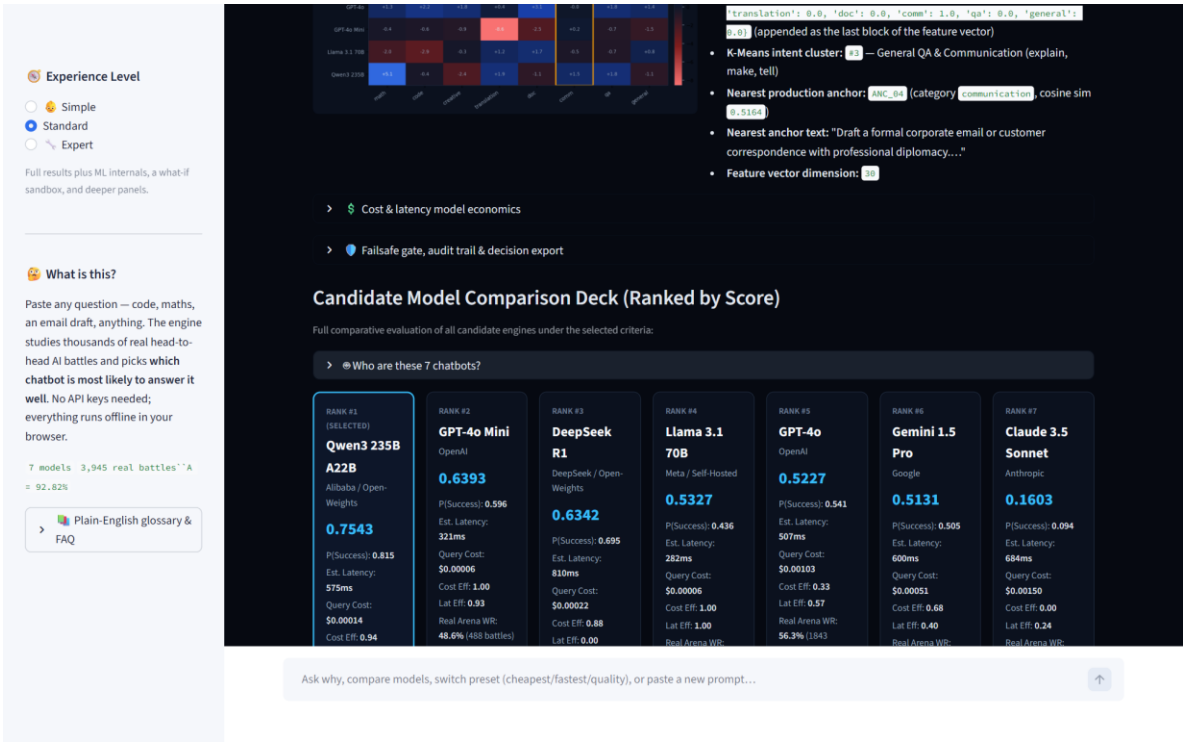


Figure 4. Candidate comparison deck. All seven models with composite score, predicted success, latency, query cost, cost and latency efficiency, and the model's real arena win-rate with its battle count. The selected model is outlined.

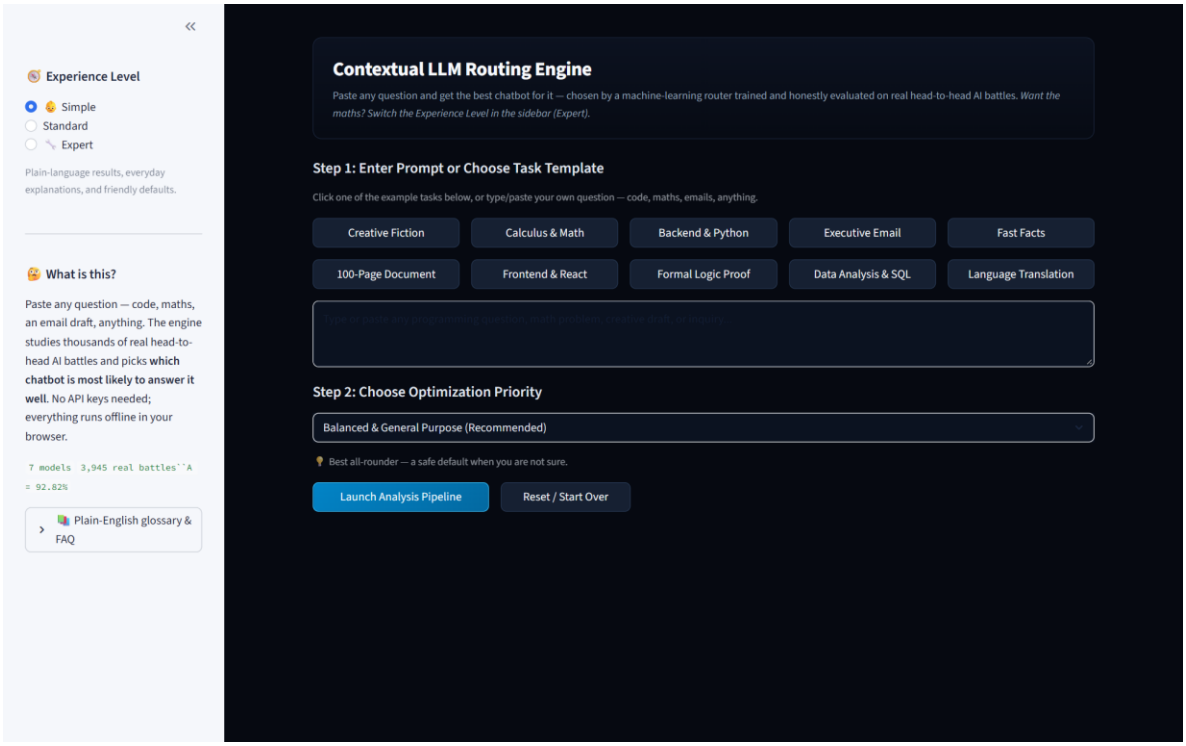


Figure 5. Simple experience level. The same decision restated without internals: an everyday-terms card, traffic-light cost and speed chips, and a one-line accuracy statement.

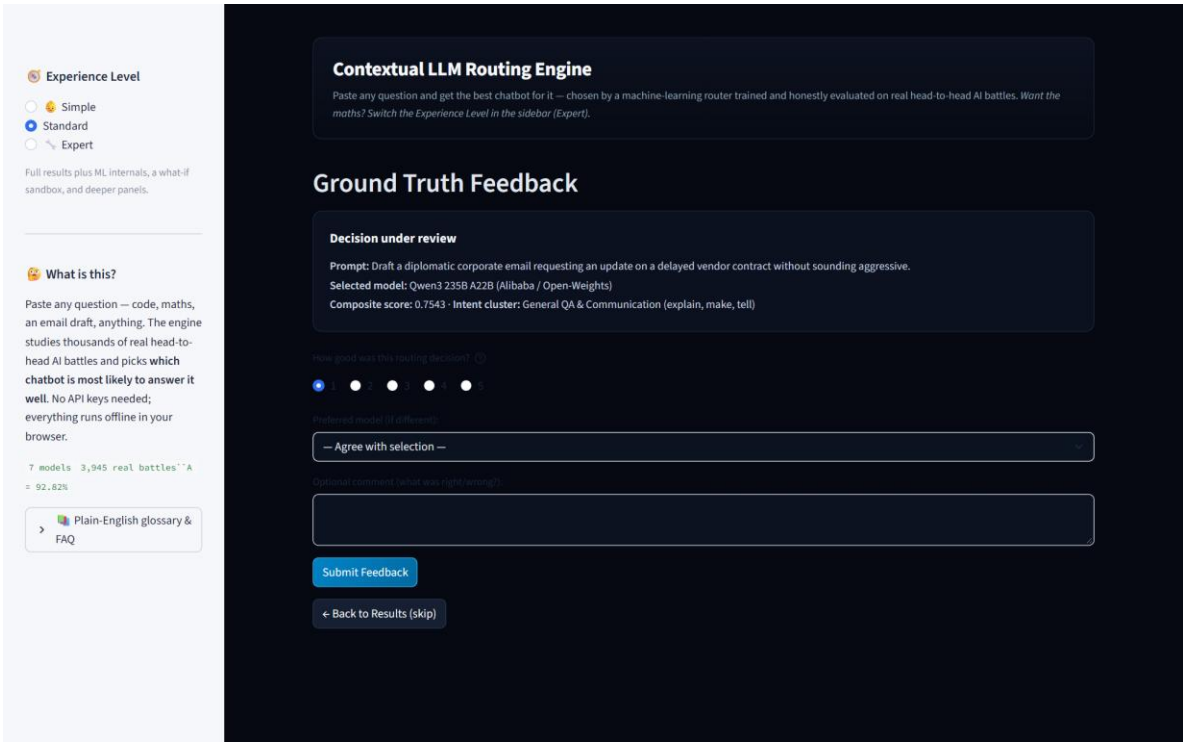


Figure 6. Feedback page. Ratings and corrections are written to disk as new rows and can be folded back into training, which is the loop that would let the router improve on an organisation's own traffic.

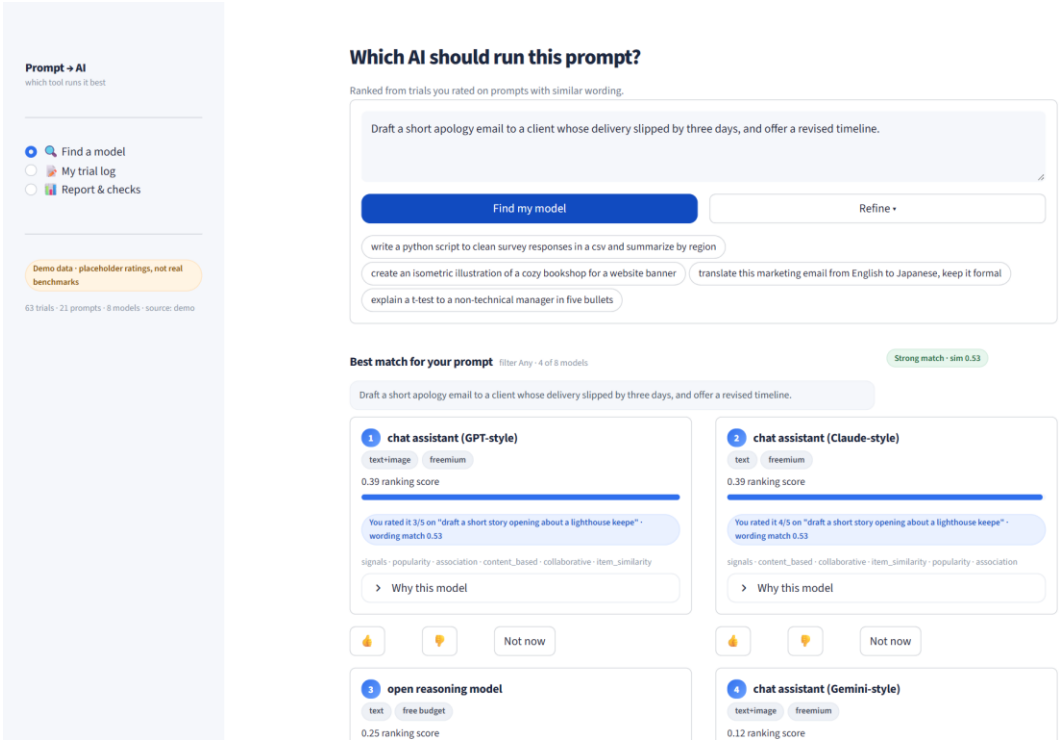


Figure 7. Prompt-domain application, results view. Each card carries one reason chip quoting an existing logged trial, a fine-print line naming the algorithms behind the ranking, thumbs up or down that writes a real 1-5 trial row, and a Not-now display filter.

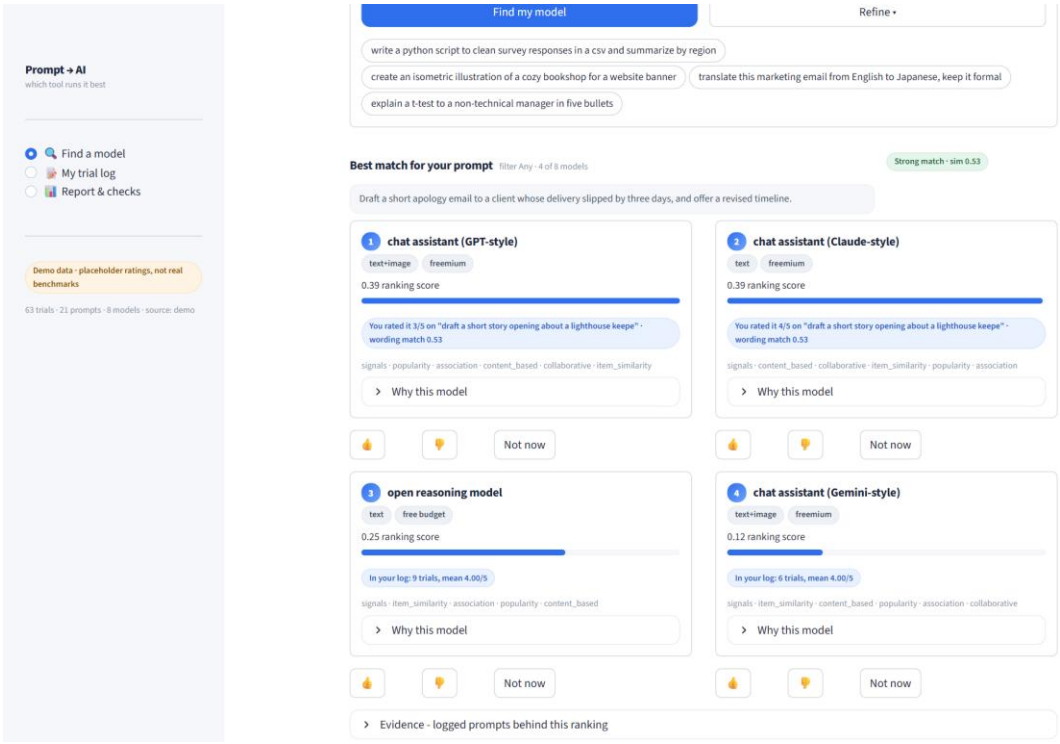


Figure 8. "How these rankings work". The explanatory prose lives behind one panel and in tooltips rather than on the working surface, so the input screen and the evidence do not compete for attention.

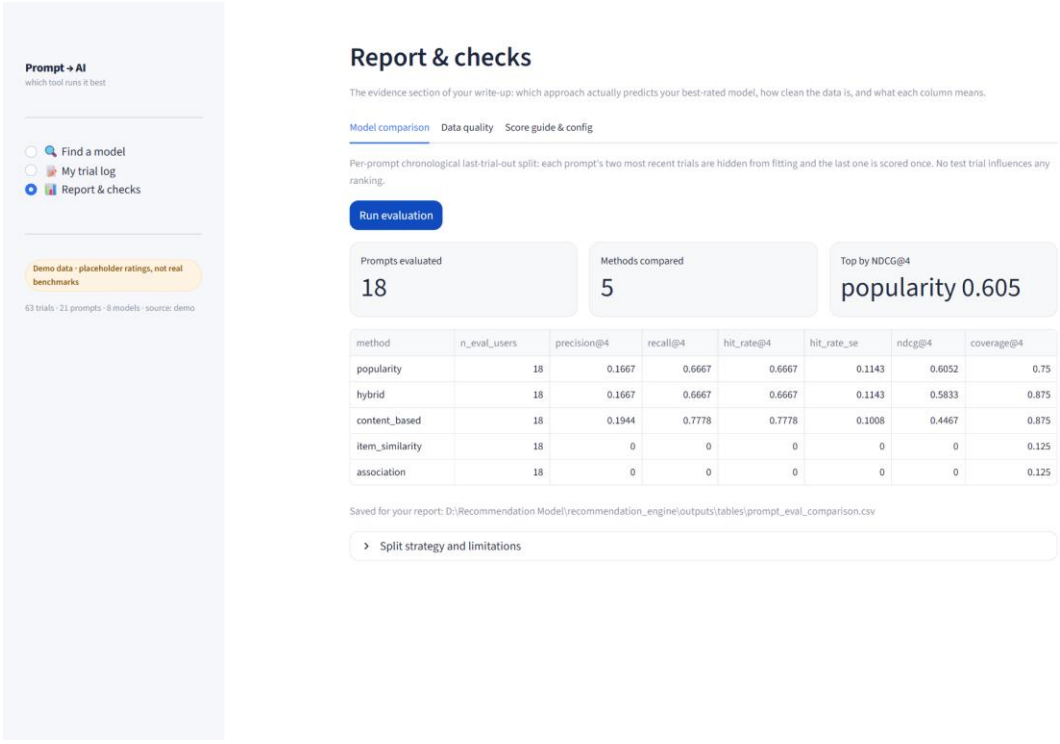


Figure 9. Report and checks. The evaluation runs on a per-prompt chronological last-trial-out split, stated on screen, so a reviewer can see the leakage control before seeing the numbers.

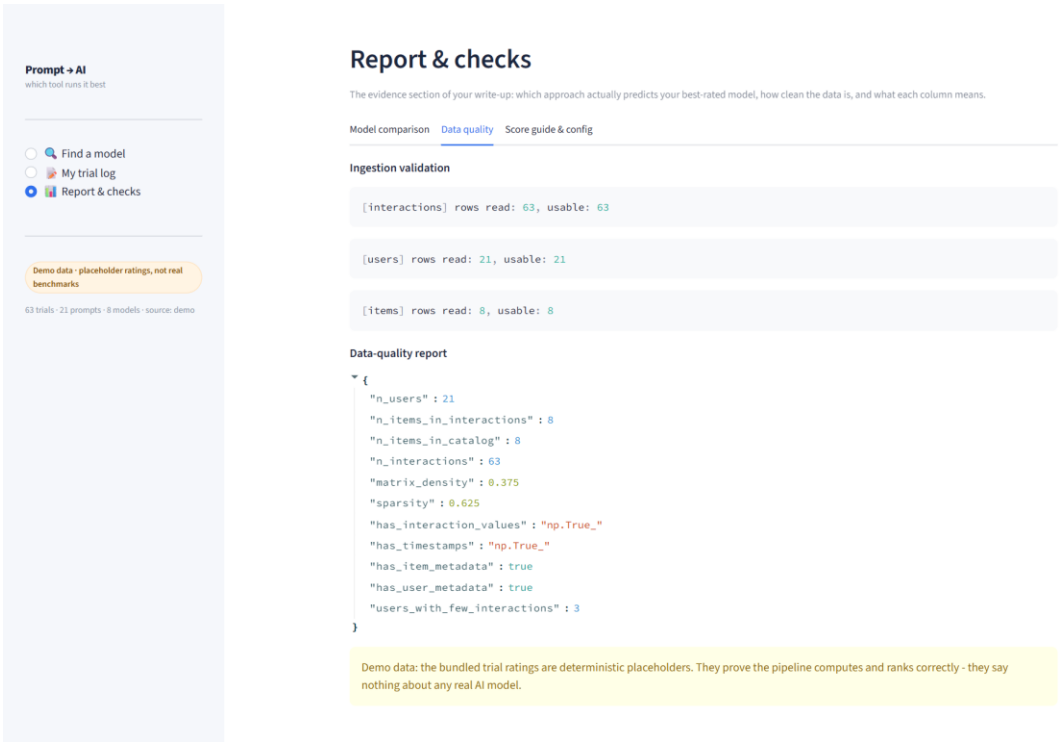


Figure 10. Data-quality tab of the same step: schema validation, sparsity, and the specific rows the recommender had to work around.

7. Limitations

- Metric B is weak. At 54.28 % the composite score separates a battle's winner from its loser only slightly better than a coin flip. The router is good at naming the right class of model for a task and poor at ranking two specific models inside that class.
- The 90 % target is met on one split, not on average. The headline 92.82 % comes from a single seed-42 held-out split; five-fold grouped cross-validation averages 88.08 % +/- 4.77, which is below target. Both are reported; the cross-validation figure is the one to plan against.
- Small cells are noisy. Translation, business-communication and long-document categories have fewer than ten held-out prompts each, and low-volume models such as DeepSeek R1 (189 training battles) carry wide win-rate uncertainty.
- Battle data is text-only arena traffic. It is not a sample of any organisation's real usage, and it reflects the preferences of an anonymous crowd on prompts they chose to submit.
- No scores are probabilities. Nothing in the system is calibrated. Predicted success is a bounded model output used for ranking, and the score-kind table in Appendix A states what each column is allowed to mean.
- Limited tuning. Hybrid weights were compared across three configurations on validation data, not optimised globally. Association rules use max-confidence aggregation rather than a full rule explorer. No deep-learning recommender is included; none was shown to be needed for this problem.
- Comparisons are point estimates with standard errors. No formal significance testing between methods yet, which would need a paired bootstrap.

8. How to run and verify

```
cd "D:\Recommendation Model\recommendation_engine"

pip install -r requirements.txt       # pandas, numpy, scipy, scikit-learn, streamlit,
                                      # matplotlib, pytest - all free, no API keys

streamlit run app_router.py           # contextual routing engine (Sections 4.2, 5.1)
streamlit run app_prompt.py           # prompt to model application
streamlit run app.py                  # generic domain-agnostic engine

python -m pytest tests -q                     # 141 tests, 83.9 s
python scripts/build_accuracy_report.py --with-cv    # regenerate Section 5.1 from raw data
python scripts/tune_accuracy.py               # logged hyper-parameter grid search
python run_demo_outputs.py                    # tables, figures, logs for the generic engine
python run_prompt_demo_outputs.py             # same for the prompt domain
python scripts/build_arena_dataset.py             # rebuild the 3,945-battle dataset
```

Requirements are free and open-source only. There is no cloud dependency, no paid API and no LLM call anywhere in the system; the in-app assistant that answers “why this model” is a local rule-based component. The screenshots in Section 6 were captured by driving the running applications with scripts/capture_ui_screenshots.py, which reported zero browser console errors across the walk-through. Test results are in docs/test_results.md with raw output in outputs/logs/pytest_verbose.txt.

Appendix A - what each score is allowed to mean

score_kind	Definition	Is it a probability?
frequency_count	Distinct users who interacted with the item	No
similarity	TF-IDF or co-occurrence cosine on [0, 1]	No
ranking_score	Latent-factor reconstruction value	No
confidence	Empirical co-occurrence rule confidence	No
normalized_ranking_score	Weighted blend of per-source rank scores	No

Appendix B - data mapping for the prompt domain

Generic role	Prompt-domain column	Notes
user_id	prompt_id	A prompt is the requesting entity
item_id	model_id	Eight models in the demo catalogue, seven in the router
interaction_value	quality_score	1-5 rating, aggregated by mean
timestamp	timestamp	Drives the chronological last-trial-out split
user attributes	task_category, language	Categorical

item attributes	modality, pricing, best_for, description	Two categorical, two free-text

Changing the domain means changing this mapping, the thresholds in config/prompt_config.json and the features; the core engine is untouched. That path is documented in docs/customization_guide.md.

Appendix C - project contents

app.py, app_prompt.py, app_router.py (three Streamlit front ends); src/engine.py (load, build, recommend facade); src/data/, src/features/, src/recommenders/, src/ranking/, src/selection/, src/evaluation/, src/statistics/, src/fallback/, src/reporting/, src/utils/; src/routing/ (classifier ensemble, dual-metric evaluation, chat assistant, UI panels, charts); src/domain/ (prompt matcher, trial log); config/, data/, scripts/, tests/, outputs/, docs/.